Tutorial 9

Modern Java

Objective

- To equip students with practical skills in modern Java programming—including generics, Collections, streams, lambda expressions, enhanced for-loops, default methods, and enhanced switch—to write cleaner, safer, and more expressive Java code.

Use Java Generics effectively

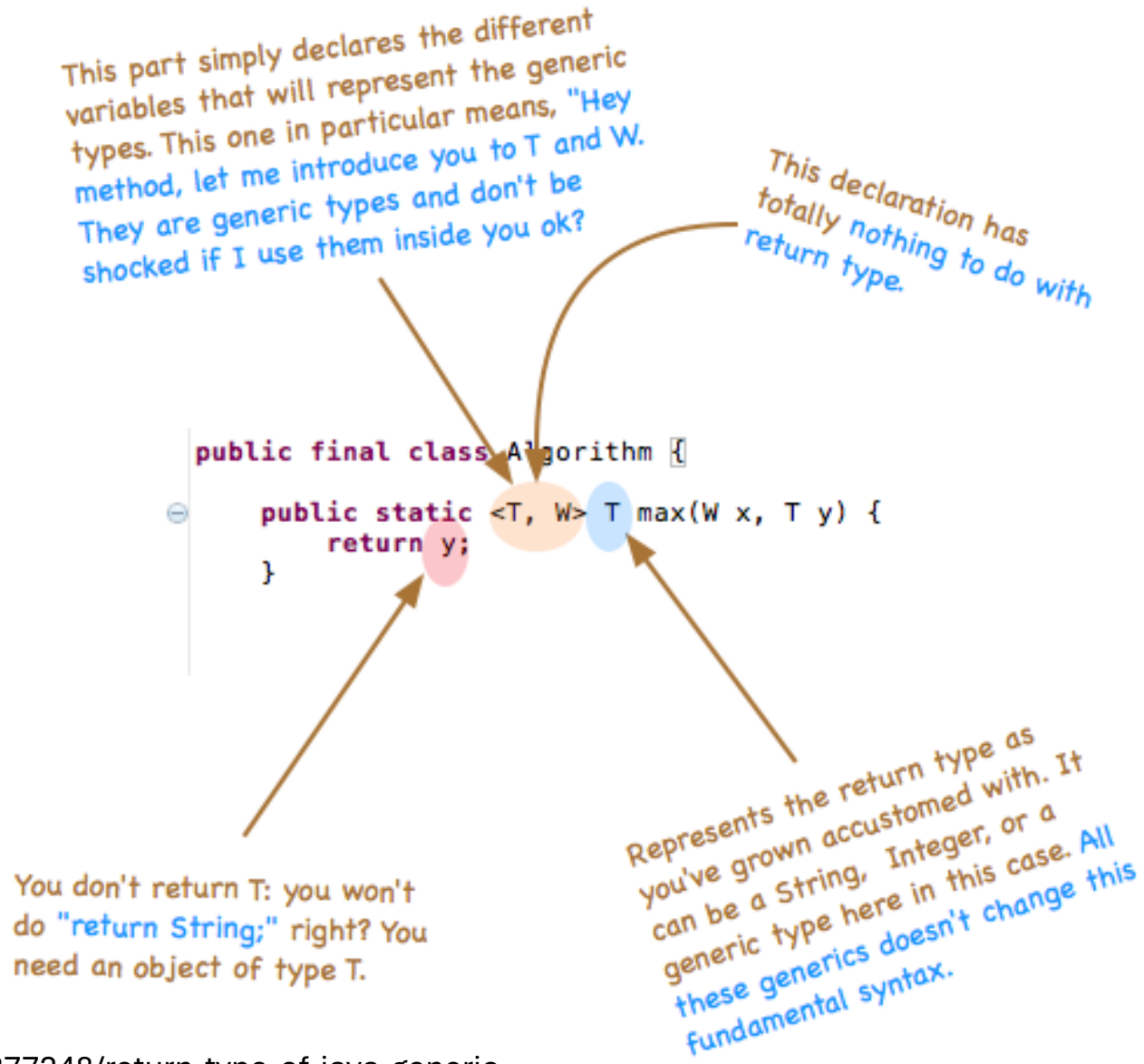
- Create and use **generic methods** that work for any reference type.
- Understand the **limitations with primitive types** and why wrapper classes are needed.

Q1

Write a generic method printArray that takes an array of **any type** and prints its elements. Test your method with arrays of Integer, String, and Double.

```
public class Main {

public static void printArray(int[] arr) {
for (int element : arr) {
System.out.print(element + " ");}
System.out.println();

public static void main(String[] args) {
int[] numbers = {10, 20, 30, 40, 50};
System.out.print("Array elements: ");
printArray(numbers);}}
```

Breaking Down a Generic Method



https://stackoverflow.com/questions/11377248/return-type-of-java-generic-methods

Which of the following correctly declares a generic method in Java?

Rule: `static` comes BEFORE the type parameter `<T>`, and `<T>` comes BEFORE the return type.

```
[modifiers] <type-parameters> return-type method-name(...)
```

Q1

Write a generic method printArray that takes an array of **any type** and prints its elements. Test your method with arrays of Integer, String, and Double.

```java
public class Main {
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.print(element + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Integer[] intArray = {1, 2, 3, 4, 5};
        String[] strArray = {"Hello", "World"};
        Double[] doubleArray = {1.1, 2.2, 3.3};

        printArray(intArray);    // Output: 1 2 3 4 5
        printArray(strArray);    // Output: Hello World
        printArray(doubleArray); // Output: 1.1 2.2 3.3
    }
}
```

Metaphor — “Universal Tools vs. Special Tools”

- **Generics = A universal screwdriver**
- A generic method is like a **universal screwdriver handle** that can accept different heads:
- Philips head
- Flat head
- Hex head
- The handle is generic; the type of head is determined when used.

Real Application — Building a “Flexible Data Export” System

- A real system (e.g., NTU FYP system) needs to export a list of:
  - Students
  - Projects
  - Supervisors
  - Schedules
  - Evaluations

Without generics, you need:

- exportProjectList(List<Project>)
- exportScheduleList(List<Schedule>)
- exportEvaluationList(List<Evaluation>)
- exportUserList(List<User>)--upCasting
  - exportStudentList(List<Student>)
  - exportSupervisorList(List<Supervisor>)

This leads to repeated code, high maintenance, and human error.

Generic solution

```
public <T> void exportList(List<T> items) {
    for (T item : items) {
        System.out.println(item.toString());
    }
}
```

Now one method works for:
Students, projects, supervisors, schedules, even custom types like Comments or Feedback
This is powerful for real enterprise systems because it reduces:
Duplication
Bugs
code size
future maintenance cost

What happens if you try to pass a primitive array (e.g.,int[]) to the printArray method? Why?

- Because Java generics operate at runtime using **Object references**.
- Primitive types (int, double):
- are stored **directly in memory**
- not as Object references
  → cannot fit into the runtime mechanism of generics.

Primitive types = tools without an adapter
Primitive types (int) cannot fit directly into the universal tool handle.

Wrapper classes = adapter that makes them compatible

- List<int> numbers; // ❌ Compile error: int is not a valid generic type

```
List<Integer> numbers = new ArrayList<>();
numbers.add(10);      // auto-boxing: int → Integer
numbers.add(20);
numbers.add(30);

for (Integer n : numbers) {
    System.out.println(n);
}
```

What happens if you try to pass a primitive array (e.g.,int[]) to the printArray method? Why?

- Wrapper classes (Integer, Double):
  - become full-fledged Objects
  - can be stored in generic collections
    → solve the compatibility problem.

Q2

a) Use the Collections utility class to perform the following operations on a List<Integer>:

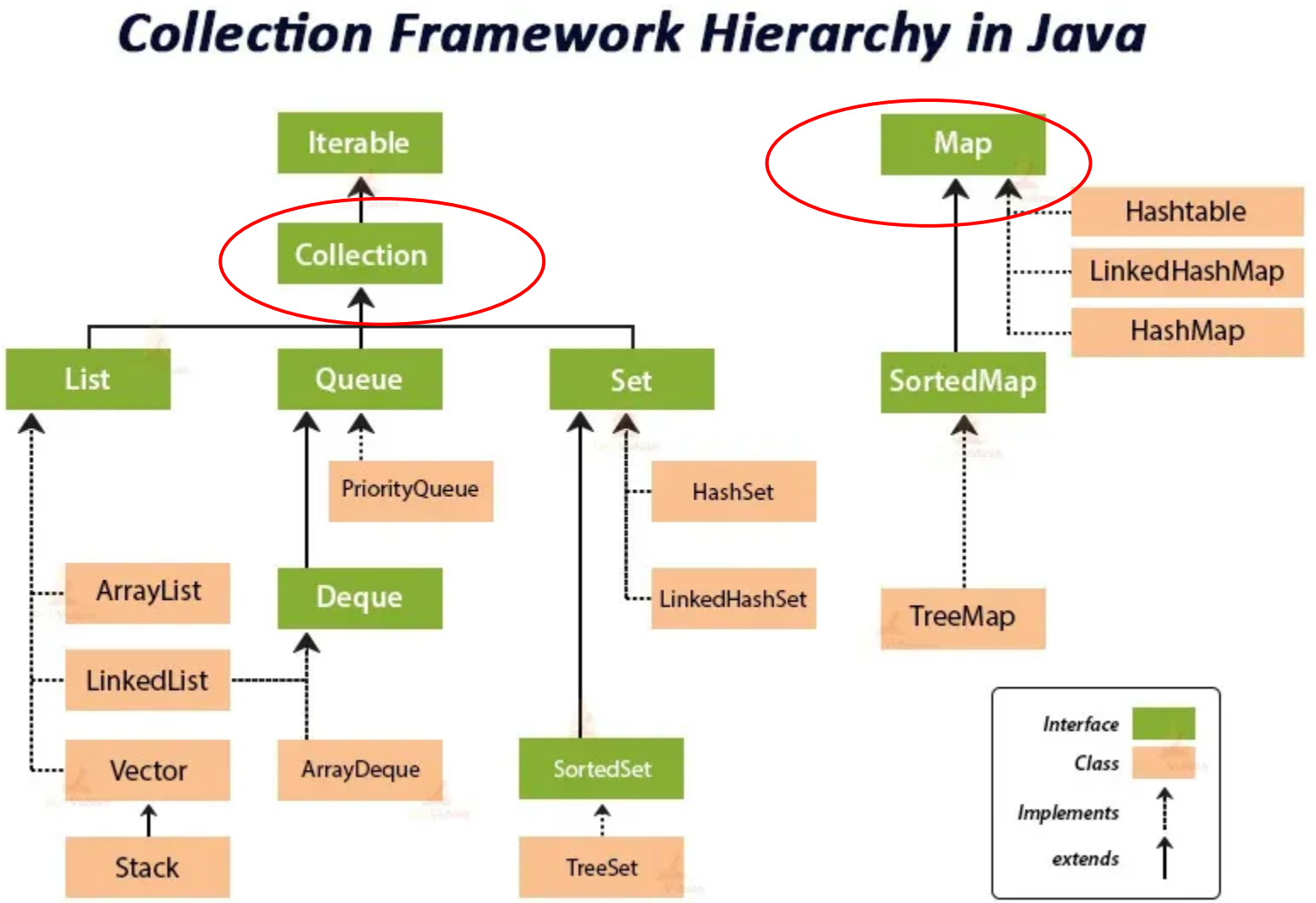
- Sort the list in descending order.
- Shuffle the list.
- Find the maximum and minimum values in the list.

b) How does the sort method handle null values in the list?

c) What is the difference between Collections.sort() and List.sort()?

https://docs.oracle.com/javase/8/docs/api/java/util/Collections.html

Java Collections Framework Hierarchy



🧠 Summary Table

Feature	`Collection`	`Collections`
Type	Interface	Utility class
Package	`java.util`	`java.util`
Purpose	Represents a group of objects	Provides static utility methods
Instantiable?	❌ (must use implementing classes)	✅ (but only contains static methods)
Examples	`List`, `Set`, `Queue`	`Collections.sort()`, `.reverse()`

Why Use the Collections Utility Class?

- Saves time (no need to write your own sorting/shuffling code)
- Prevents bugs
- Ensures consistent, well-tested behavior
- Widely used in real systems (dashboards, ranking, schedules, timetables)

```java
import java.util.Arrays;
import java.util.Collections;
import java.util.List;


public class Main {
public static void main(String[] args) {
List<Integer> numbers = Arrays.asList(10, 5, 8, 3, 1);

// Sort in descending order
Collections.sort(numbers, Collections.reverseOrder());
System.out.println("Sorted in descending order: " + numbers);

// Shuffle the list
Collections.shuffle(numbers);
System.out.println("Shuffled list: " + numbers);

System.out.println("Max: " + Collections.max(numbers));
System.out.println("Min: " + Collections.min(numbers)); }}
```

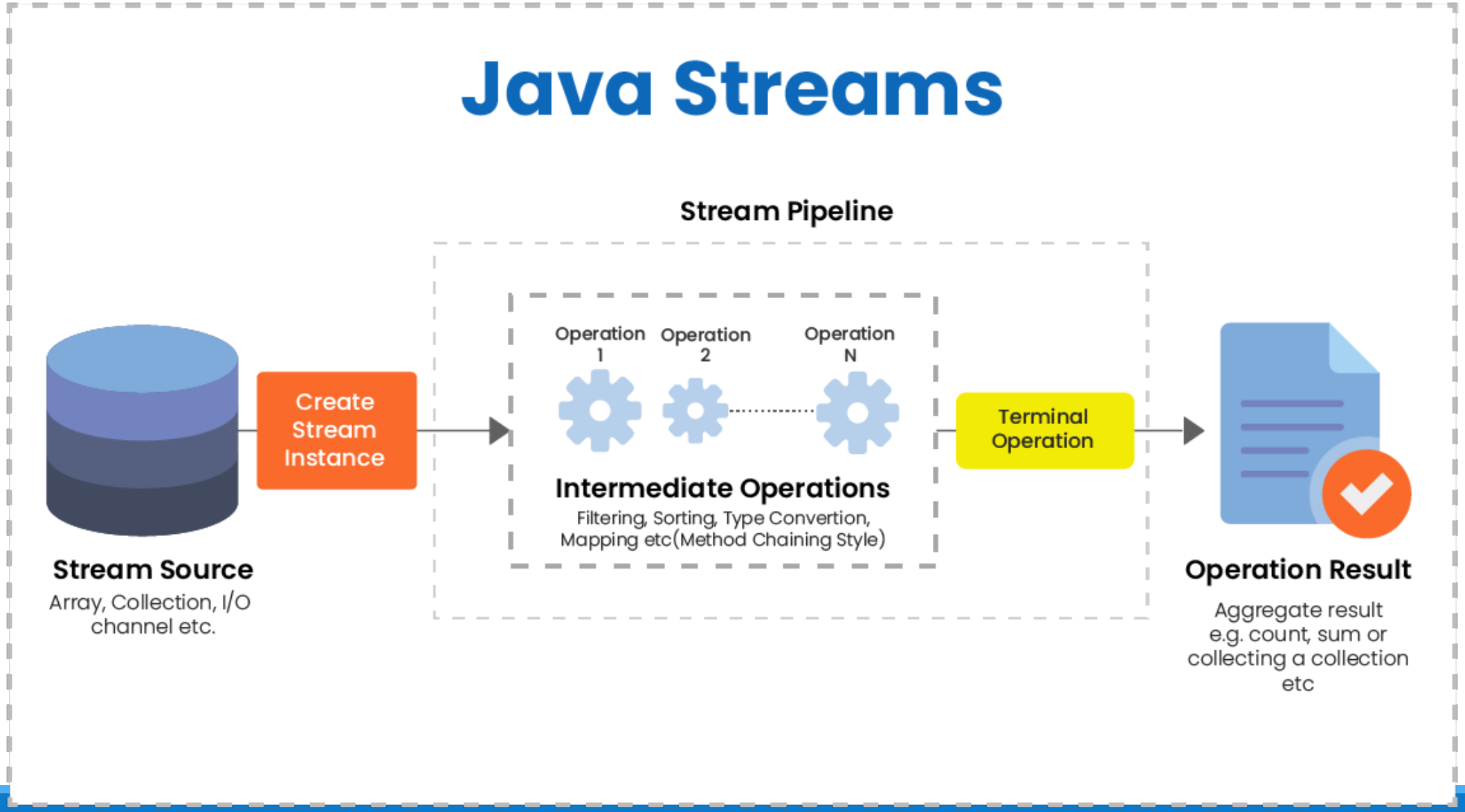
b) Collections.sort() throws a NullPointerException if the list contains null values.

c) Collections.sort() is a static method, while List.sort() is an instance method. Both can be used interchangeably, but List.sort() is more object-oriented.

Method	Type	Style	Recommendation
Collections.sort(list)	static utility method	old-style Java	✅ still works, but less modern
list.sort(comparator)	instance method	modern Java (Java 8+)	✅ preferred in modern code

```java
import java.util.Arrays;
import java.util.Collections;
import java.util.List;


public class Main {
public static void main(String[] args) {
List<Integer> numbers = Arrays.asList(10, 5, 8, 3, 1);

// Sort in descending order
numbers.sort(Collections.reverseOrder());
System.out.println("Sorted in descending order: " + numbers);

// Shuffle the list
Collections.shuffle(numbers);
System.out.println("Shuffled list: " + numbers);

System.out.println("Max: " + Collections.max(numbers));
System.out.println("Min: " + Collections.min(numbers)); }}
```

What is Stream API?



```
List<String> fruits = Arrays.asList("apple", "kiwi", "pear",
"mango");

long count = fruits.stream()
        .filter(s -> s.contains("a"))
        .count();

System.out.println(count);
```

What is printed?

- “filter acts like a sieve — only items that match the condition pass through.”
- “After filter(), the stream contains only elements that meet the criteria.”
- “Elements that do not meet the condition are dropped from the stream.”

Audience Q&A

Q3

a) Given a list of strings, use the Streams API to:

- Filter out all strings with a length less than 5.
- Convert the remaining strings to uppercase.
- Collect the results into a new list.

```
List<String> list = Arrays.asList("a", "bb", "ccc");

List<Integer> result = list.stream()
        .map(s -> s.length())
        .collect(Collectors.toList());

System.out.println(result);
```

What is the output?

map() transforms each element in the stream into a new value.
For every input element, map() produces exactly one output element.

```java
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;


public class Main {
public static void main(String[] args) {
List<String> words = Arrays.asList("apple", "banana", "kiwi", "orange", "pear");


List<String> result = words.stream()
.filter(s -> s.length() >= 5) // Filter strings with length >= 5
.map(String::toUpperCase) // Convert to uppercase
.collect(Collectors.toList()); // Collect into a list


System.out.println(result); // Output: [APPLE, BANANA, ORANGE]
}
}
```

b) How would you modify the code to count the number of strings that meet the filtering criteria?

To count the number of strings that meet the criteria, use .count() instead of .collect():

```
long count = words.stream().filter(s -> s.length() >= 5).count();
System.out.println("Count: " + count);
```

Map vs flaMap

c) What is the difference between map() and flatMap() in the Streams API?

c) map() transforms each element into another object, while flatMap() flattens nested structures (e.g., List<List<T>> to List<T>).

- **map() keeps the "boxes inside boxes."**
- **flatMap() opens the inner boxes and puts everything into one long box.**
- **After flatMap(), the data becomes a flat (1D) sequence.**

Real-life Example: NTU Course System — Students Taking Multiple Courses

```
class Student {
    String name;
    List<String> courses;
}
```

```
List<Student> students = List.of(
   new Student("Alice", List.of("SC2002", "SC2003")),
   new Student("Bob",   List.of("SC2002")),
   new Student("Cindy", List.of("SC1007", "SC2002",
"SC3000"))
);
```

```
List<String> allCourses = students.stream()
      .flatMap(s -> s.getCourses().stream())
      .collect(Collectors.toList());
```

[SC2002, SC2003, SC2002, SC1007, SC2002, SC3000]

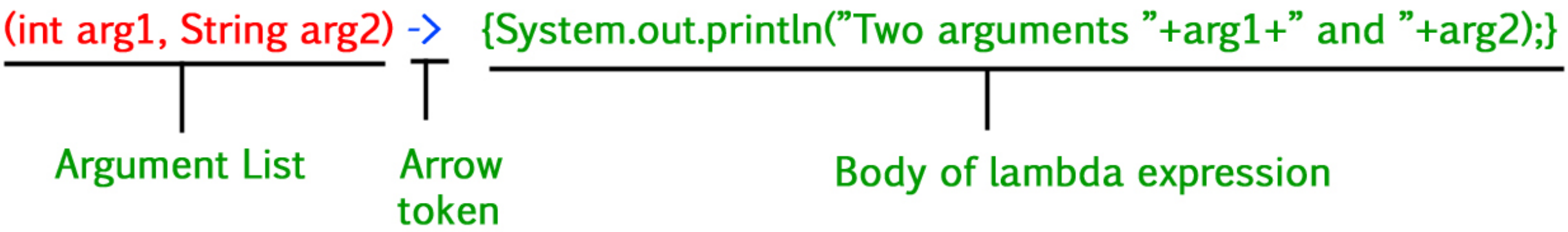
Q4

Rewrite the following anonymous class using a lambda expression:

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hello, World!");
    }
};
```

Lambda Expressions – Basic Syntax

(arguments) -> { body }



```java
public class Main {
    public static void main(String[] args) {
        Runnable r1 = new Runnable() {
            @Override
            public void run() {
                System.out.println("anonymous class!");
            }
        };
        r1.run();

        Runnable r2 = () -> System.out.println("Lambda!");
        r2.run();

    }
}
```

Can all functional interfaces be replaced with lambda expressions? Why or why not?

- Yes, all functional interfaces (interfaces with a single abstract method) can be replaced with lambda expressions.

What are the benefits of using lambda expressions over anonymous classes?

Lambda expressions are more concise, improve readability, and reduce boilerplate code compared to anonymous classes.

- **Lambdas** in Java allow you to:
  - Write **shorter, clearer** code compared to anonymous classes.
  - Pass **behavior** (functions) around as objects.
  - Use **functional interfaces** to perform operations on collections, handle callbacks, and more.
- They are a cornerstone of Java's functional programming features and greatly simplify many common tasks involving single-method interfaces.

Q5

a) Given an array of integers, use the enhanced for loop to calculate the sum of all even numbers in the array.
b) How does the enhanced for loop differ from the traditional for loop?
c) Can you use the enhanced for loop with custom objects? If so, how?

```
public class Main {
public static void main(String[] args) {
int[] numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int sum = 0;


for (int num : numbers) {
if (num % 2 == 0) {
sum += num;
}}


System.out.println("Sum of even numbers: " + sum); // Output: 30
} }
```

The enhanced (modern) for loop is generally better suited for read-only operations because:

Improved Readability: Its concise syntax makes the code cleaner and easier to understand.

Reduced Error Potential: There's no need to manage index variables, which minimizes common errors like off-by-one mistakes.

Direct Element Access: It directly accesses each element in the collection or array, which is ideal when you only need to process or display values.

However, if you need to modify elements or require the element index for more complex operations, the classic for loop might be a better choice.

Q6

a) Create an interface Vehicle with a default method start() that prints "Vehicle started." Then, create a class Car that implements Vehicle and overrides the start() method to print "Car started."

b) What happens if a class implements two interfaces with the same default method? How can this conflict be resolved?

c) Why were default methods introduced in Java 8?

```
interface Vehicle {
default void start() {
System.out.println("Vehicle started."); }}

class Car implements Vehicle {
@Override
public void start() {
System.out.println("Car started."); }}
public class Main {
public static void main(String[] args) {
Vehicle car = new Car();
car.start();
}}
```

b) If a class implements two interfaces with the same default method, it must override the method to resolve the conflict.

If **two interfaces** provide a **default method with the same signature**, and a class implements both, the **compiler will throw an error**, because **Java cannot decide which one to inherit**.

```
interface A {
   default void greet() {
      System.out.println("Hello from A");
   }}

interface B {
   default void greet() {
      System.out.println("Hello from B");
   }}

public class MyClass implements A, B {
   // Compilation error: class inherits unrelated defaults for greet() from types A and B
}
```

How to Achieve Multiple Inheritance with Default Methods?

```
interface A {
default void show() {
System.out.println("Default method from A");}}

interface B {
default void show() {
System.out.println("Default method from B");}}

class C implements A, B {
@Override
public void show() {
System.out.println("Resolving conflict by overriding");
A.super.show(); // Choose method from interface A}}
```

c) Default methods were introduced to allow adding new methods to interfaces without breaking existing implementations.

🧠 Summary

Without default method	With default method
All implementing classes break when a new method is added	No changes needed; default method provides fallback
Forces code duplication in every implementation	Encourages code reuse

Q7

a) Rewrite the following switch statement using the enhanced switch syntax:

```
int day = 3;
String dayType;
switch (day) {
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
        dayType = "Weekday";
        break;
    case 6:
    case 7:
        dayType = "Weekend";
        break;
    default:
        dayType = "Invalid day";
}
```

```
public class Main {
public static void main(String[] args) {
int day = 3;
String dayType = switch (day) {
case 1, 2, 3, 4, 5 -> "Weekday";
case 6, 7 -> "Weekend";
default -> "Invalid day";
};


System.out.println(dayType); // Output: Weekday
}
}
```

Q7

a) What are the advantages of using the enhanced switch over the traditional switch statement?

b) Can the enhanced switch be used with non-primitive types (e.g., String)? If so, how?

b) The enhanced switch is more concise, supports multiple cases in a single line, and eliminates the need for break statements.

c) Yes, the enhanced switch can be used with non-primitive types like String.

```
public class Main {
public static void main(String[] args) {
String day = "MONDAY";


String typeOfDay = switch (day.toUpperCase()) {
case "MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY" -> "Weekday";
case "SATURDAY", "SUNDAY" -> "Weekend";
default -> "Invalid day";
};


System.out.println("It's a: " + typeOfDay);
}
}
```